

Sistema de gestión Taller de colisión de vehículos

Guía de referencia para codificación

Clases · Atributos · Tipos de datos · Métodos · Reglas de negocio

Stack: React + Spring Boot + MySQL

Versión sincronizada con el código — 5 de septiembre de 2026

- Las longitudes de las columnas **VARCHAR** ya no son 255 genérico: se ajustaron al dato real del taller. Esta guía refleja lo que está anotado hoy en las entidades JPA.
- Los recuadros como este marcan dónde la versión anterior de la guía quedó desactualizada, para que se note qué cambió y por qué.
- Regla de precedencia: **manda el código**. Si una entidad y esta guía difieren, se corrige la guía (y se regenera desde **docs/generar_guia.py**), no la entidad.

Convenciones usadas en este documento

[0..1] = campo nullable (opcional) | PK = llave primaria | FK = llave foránea | UNIQUE = restricción de unicidad

Los tipos Long/Int/Short son Java; la columna MySQL muestra el equivalente en el esquema.

La columna **Nombre** usa el nombre del atributo Java; cuando la columna SQL no coincide, se indica entre paréntesis.

Cambios de esta versión

Refactor del modelo del 5 de septiembre de 2026. Afecta a las 17 entidades, así que se documenta una sola vez aquí en vez de repetirlo en cada clase.

Qué cambió	Antes	Ahora
Longitudes VARCHAR	255 para todo campo de texto	Ajustadas al dato real (ver cada clase). Ej: placa 6, documento 10, celular 10, nota 500, url de imagen 2083
Lombok	@Getter / @Setter / @NoArgsConstructor	Sin Lombok. Cada entidad escribe a mano constructor vacío, constructor completo, getters y setters
Timestamps	@PrePersist / @PreUpdate en la entidad	Callbacks eliminados. createAt / updateAt los asigna el Service (siguen siendo NOT NULL)
Métodos de negocio	cambiarEtapa(), asignarTecnico(), calcularDiasEnTaller(), marcarCargadaCesvi(), marcarEnviada()/marcarFallida() dentro de la entidad	Entidades anémicas: no tienen comportamiento. Toda la lógica vive en la capa de servicio
CHECK de gravedad	CHECK ck_detalle_gravedad en el DDL de ValoracionDetalle	Eliminado del esquema. La regla sigue vigente pero la valida el Service
Índice de placa	INDEX idx_vehiculo_placa	Eliminado. Pendiente reponerlo: la búsqueda por placa es la consulta más frecuente del sistema
Cascadas	Valoracion.detalles y Valoracion.imagenes con CASCADE ALL + orphanRemoval	Sin cascada: el Service borra los hijos por su propio repositorio. Sólo Observacion.imagenes conserva la cascada
Columnas de Cliente	documento, nombre_completo, celular, correo	Con sufijo de tabla: documento_cliente, nombre_cliente, celular_cliente, correo_cliente (los atributos Java no cambian)

PENDIENTES QUE DEJÓ EL REFACTOR

- **Usuario** se quedó sin constructores, getters ni setters: se le quitó Lombok y no se repusieron. Compila porque todavía nada la consume.
- Faltan longitudes explícitas en **Usuario** (nombre, email, passwordHash) y en **Notificaciones** (eventoDisparador, canal, contenidoEnviado, errorDetalle). Quedan en el default 255.
- Nadie asigna createAt / updateAt hoy: hay que decidir entre hacerlo en el Service, reponer los callbacks JPA o usar auditoría de Spring Data.
- **celular** con length 10 no admite el formato E.164 (+573001234567, 13 caracteres) que exige la API de WhatsApp.
- Las longitudes nuevas son estrictas: MySQL trunca o falla si el dato entra más largo. Falta validación con Bean Validation (@Size / @Pattern) en los DTOs.

Tipos de datos — Java → MySQL

Referencia rápida para mapear los tipos del modelo Java al esquema MySQL al crear las entidades JPA y las migraciones.

Tipo Java	MySQL	Long./Prec.	Uso típico
Long	BIGINT	20	Llaves primarias y foráneas
Integer	INT	11	Enteros medianos (orden de etapa)
Short	SMALLINT	6	Enteros pequeños (año del vehículo)
Byte	TINYINT	3	Valores muy pequeños (días estimados, máx. 127)
boolean	BOOLEAN	1	true/false → TINYINT(1) en MySQL
String	VARCHAR	según campo	Ya no se usa 255 por defecto: cada columna declara su ancho real
LocalDate	DATE	—	Solo fecha: yyyy-MM-dd
LocalDateTime	DATETIME	—	Fecha + hora: yyyy-MM-dd HH:mm:ss
BigDecimal	DECIMAL	12,2	Montos y costos
byte[]	BLOB	—	Archivos binarios (no se usa hoy)
String (JSON)	JSON	—	Arrays JSON como imagenesEnviadas

Sobre las longitudes

- Los **enums** van siempre con **length = 20** y **@JdbcTypeCode(SqlTypes.VARCHAR)**. Sin ese JdbcTypeCode, Hibernate 7 genera un tipo ENUM(...) nativo de MySQL en vez de VARCHAR(20), lo que obliga a un ALTER TABLE cada vez que se agrega un valor al enum.
- **documento** y **celular** son String aunque parezcan numéricos: un documento puede llevar ceros a la izquierda o dígito de verificación.
- Ninguna columna guarda imágenes como BLOB. Se guarda la URL pública https porque WhatsApp y CESVI descargan el archivo, no lo reciben.

Enumeraciones

Tipos enum usados como atributos en las clases. Viven en el subpaquete **modelo.Enums** y son clases Java puras, sin anotaciones. La anotación va en la entidad que los usa: `@Enumerated(EnumType.STRING)` + `@JdbcTypeCode(SqlTypes.VARCHAR)` + `@Column(length = 20)`.

Enum	Valores	Descripción y uso
RolUsuario	ADMIN ASESOR GERENTE TECNICO	Controla qué módulos ve cada usuario. GERENTE: sin admin de usuarios. ASESOR: operación diaria.
Estado	ACTIVA CERRADA CANCELADA	Estado de la OrdenReparacion. Un boolean no alcanza para este caso.
EstadoEnvio	PENDIENTE ENVIADO FALLIDO	Estado del mensaje en Notificaciones. Permite reintentos y auditoría.
Accion	REPARACION SUSTITUCION	Qué se hace con la pieza en ValoracionDetalle.
Gravedad	LEVE MEDIO FUERTE	Solo aplica si Accion = REPARACION. Si es SUSTITUCION, este campo es NULL.
Ubicacion	EN_TALLER FUERA_DE_TALLER	Ubicación del vehículo al momento de la Observacion.
Repuestos	COMPLETOS PENDIENTES	Estado de los repuestos al dejar la Observacion.

Catálogos

Sobre las tablas de MÉTODOS

- Las firmas que se listan en cada clase describen la **API de la capa de Service/Repository**, no métodos de la entidad JPA.
- Tras el refactor, **ninguna entidad tiene métodos de negocio**. Lo que antes vivía en OrdenReparacion, Valoracion y Notificaciones se reescribe en el Service correspondiente.

Cliente

Propietario del vehículo. Contiene todos sus datos de contacto directamente (Opción A: sin clase Persona).

Cambios respecto a la versión anterior

- Longitudes: documento 255 → **10**, nombreCompleto 255 → **100**, celular 255 → **10**, correo 255 → **100**.
- Las columnas SQL llevan sufijo **_cliente**. Es la única entidad del modelo con ese criterio; el resto usa el nombre pelado.
- celular** ya no guarda formato E.164 (+57...): con length 10 no cabe. El Service tiene que anteponer el prefijo al armar el mensaje de WhatsApp, o hay que subir la columna a 15.

ATRIBUTOS

Nombre	Java	MySQL	Long.	Restricción	Descripción
idCliente	Long	BIGINT	20	PK	Identificador único autoincremental
documento (documento_cliente)	String	VARCHAR	10	NOT NULL	Cédula del cliente. 10 no alcanza para NIT con dígito de verificación
nombreCompleto (nombre_cliente)	String	VARCHAR	100	NOT NULL	Nombre y apellidos completos
celular (celular_cliente)	String	VARCHAR	10	NOT NULL	Móvil colombiano sin prefijo, ej: 3001234567
correo (correo_cliente)	String	VARCHAR	100	[0..1]	Correo electrónico, nullable
createAt	LocalDateTime	DATETIME	—	NOT NULL	Fecha de creación. La asigna el Service
updateAt	LocalDateTime	DATETIME	—	NOT NULL	Última modificación. La asigna el Service

MÉTODOS — API de Service/Repository

Firma	Retorno	Descripción
createCliente(Cliente)	Cliente	Crea nuevo cliente
updateCliente(Cliente)	Cliente	Actualiza datos del cliente
getClienteById(idCliente)	Cliente	Busca por ID
getClienteByDocumento(documento)	Cliente	Busca por cédula/NIT
getAllCliente()	List<Cliente>	Lista todos los clientes
buscarPorPlaca(placa)	Cliente	Busca cliente dueño de la placa
deleteClienteById(id)	void	Elimina cliente por ID

RELACIONES

- Cliente 1 → N Vehiculo (mapeada: Cliente.vehiculos)
- Cliente 1 → N OrdenReparacion (sólo el lado dueño)
- Cliente 1 → N Notificaciones (receptor de mensajes) (sólo el lado dueño; se consulta por repositorio)

Tecnico

Trabajador del taller. Tabla independiente (Opción A). Puede estar vinculado a un Usuario si necesita login. En el MVP no tiene acceso web.

Cambios respecto a la versión anterior

- Longitudes: documento 255 → **10**, nombreCompleto 255 → **200**, celular 255 → **10**, correo 255 → **50**, especialidad 255 → **100**.
- Las columnas **no** llevan sufijo de tabla, a diferencia de Cliente.
- El nombre de la tabla es **tecnico** (en una versión anterior decía «tenico»).

ATRIBUTOS

Nombre	Java	MySQL	Long.	Restricción	Descripción
idTecnico	Long	BIGINT	20	PK	Identificador único autoincremental
documento	String	VARCHAR	10	NOT NULL, UNIQUE	Cédula del técnico
nombreCompleto	String	VARCHAR	200	NOT NULL	Nombre y apellidos
celular	String	VARCHAR	10	NOT NULL	Teléfono de contacto
correo	String	VARCHAR	50	[0..1]	Correo electrónico, nullable
especialidad	String	VARCHAR	100	[0..1]	Ej: Pintura, Latonería, Electromecánica
activo	boolean	BOOLEAN	1	NOT NULL	false = técnico desactivado
createAt	LocalDateTime	DATETIME	—	NOT NULL	La asigna el Service
updateAt	LocalDateTime	DATETIME	—	NOT NULL	La asigna el Service

MÉTODOS — API de Service/Repository

Firma	Retorno	Descripción
createTecnico(Tecnico)	Tecnico	Crea técnico
getTecnicoById(id)	Tecnico	Busca por ID
updateTecnico(Tecnico)	Tecnico	Actualiza datos
getAllTecnico()	List<Tecnico>	Lista todos los técnicos
getEtapasCompletadas(desde, hasta)	List<OrdenEtapaFecha>	Base para el cálculo de pagos futuros

RELACIONES

- Tecnico referenciado desde OrdenReparacion.tecnicoResponsable (nullable)
- Tecnico referenciado desde OrdenEtapaFecha.tecnico (quién trabajó cada etapa)
- Tecnico 1 → 1 Usuario (mapeada: Tecnico.usuario, opcional)

Vehículo

Vehículo registrado en el taller. La placa es el campo de búsqueda principal de todo el sistema.

Cambios respecto a la versión anterior

- Longitudes: placa 255 → **6**, marca 255 → **20**, modelo 255 → **10**, color 255 → **50**, vin 255 → **100**.
- **Se eliminó el índice `idx_vehiculo_placa`**. Hoy la búsqueda por placa hace full scan. Conviene reponerlo (como INDEX, no UNIQUE).
- **modelo** con length 10 se queda corto si aquí va la línea completa del vehículo (ej: «Sander Stepway»). Verificar el uso real.

ATRIBUTOS

Nombre	Java	MySQL	Long.	Restricción	Descripción
idVehiculo	Long	BIGINT	20	PK	Identificador único
cliente (id_cliente)	Long	BIGINT	20	FK NOT NULL	Referencia al propietario
placa	String	VARCHAR	6	NOT NULL	Placa colombiana sin guion: ABC123 o ABC12D. Ya no lleva INDEX
marca	String	VARCHAR	20	NOT NULL	Marca del vehículo
modelo	String	VARCHAR	10	NOT NULL	Línea o modelo
anio	Short	SMALLINT	6	NOT NULL	Año de fabricación
color	String	VARCHAR	50	[0..1]	Color del vehículo
vin	String	VARCHAR	100	[0..1]	Número de identificación vehicular

MÉTODOS — API de Service/Repository

Firma	Retorno	Descripción
createVehiculo(Vehiculo)	Vehiculo	Registra vehículo nuevo
getVehiculoById(idVehiculo)	Vehiculo	Busca por ID
getVehiculoByPlaca(placa)	Vehiculo	Busca por placa — uso frecuente
updateVehiculo(Vehiculo)	Vehiculo	Actualiza datos del vehículo
getAllVehiculo()	List<Vehiculo>	Lista todos los vehículos
deleteVehiculoById(id)	void	Elimina vehículo

REGLAS DE NEGOCIO

- El Service debe normalizar la placa (mayúsculas, sin espacios ni guion) antes de guardar: con length 6 cualquier caracter extra se trunca.
- La placa **no** es UNIQUE: un vehículo puede volver al taller y las placas se reasignan.

RELACIONES

- Vehiculo N → 1 Cliente
- Vehiculo 1 → N OrdenReparacion (mapeada: Vehiculo.ordenesReparacion)

Etapas

Catálogo configurable del flujo de reparación. NO se hardcodea en el backend. Flujo: Ingreso a cotizar → Desarme → Latonería → Bancada → Electromecánica → Pintura → Armado → Control de calidad → Listo para entregar → Entregado.

Cambios respecto a la versión anterior

- La clase se llama **Etapas** (plural), no Etapa.
- La columna del nombre es **nombre_etapa**, con length 255 → **100**.
- **orden** es Integer (wrapper), no int primitivo.

ATRIBUTOS

Nombre	Java	MySQL	Long.	Restricción	Descripción
idEtapa	Long	BIGINT	20	PK	Identificador único
nombre (nombre_etapa)	String	VARCHAR	100	NOT NULL	Nombre de la etapa, ej: Desarme
orden	Integer	INT	11	NOT NULL	Posición en el flujo (1, 2, 3...). Sin @GeneratedValue: es un valor de negocio reordenable por el admin
descripcion	String	VARCHAR	255	[0..1]	Descripción opcional de la etapa

MÉTODOS — API de Service/Repository

Firma	Retorno	Descripción
createEtapa(Etapas)	Etapas	Crea nueva etapa en el catálogo
getEtapaById(idEtapa)	Etapas	Busca por ID
updateEtapa(Etapas)	Etapas	Actualiza datos de la etapa
getAllEtapa()	List<Etapas>	Lista todas las etapas ordenadas
deleteEtapa(idEtapa)	void	Elimina etapa del catálogo

RELACIONES

- Etapas referenciada desde OrdenReparacion.etapaActual
- Etapas referenciada desde HistorialEtapas.etapa
- Etapas referenciada desde OrdenEtapaFecha.etapa
- Etapas referenciada desde Observacion.etapa (nullable)

Usuario

Cuenta de acceso al sistema. Vinculada opcionalmente a un Técnico si ese técnico también opera la plataforma.

Atención: esta clase está incompleta

- **Usuario no tiene constructores, getters ni setters.** Se le quitó Lombok y no se repusieron a mano como en el resto del modelo. Compila hoy sólo porque ninguna clase la consume todavía.
- (*) **nombre, email y passwordHash** no declaran length, así que quedan en el default 255 de JPA. Rompe la convención del proyecto de siempre declararlo.
- Ojo con **passwordHash**: un hash BCrypt son 60 caracteres y Argon2 puede pasar de 95, así que 255 sobra — pero conviene fijarlo explícito.

ATRIBUTOS

Nombre	Java	MySQL	Long.	Restricción	Descripción
idUsuario	Long	BIGINT	20	PK	Identificador único
nombre	String	VARCHAR	255 *	NOT NULL	Nombre del usuario del sistema. Sin length explícito
email	String	VARCHAR	255 *	NOT NULL, UNIQUE	Correo para login. Sin length explícito
rol	RolUsuario	VARCHAR	20	NOT NULL	ADMIN ASESOR GERENTE TECNICO
tecnico (id_tecnico)	Long	BIGINT	20	FK [0..1]	Si el usuario es también técnico del taller
passwordHash	String	VARCHAR	255 *	NOT NULL	Contraseña encriptada (bcrypt). Sin length explícito
activo	boolean	BOOLEAN	1	NOT NULL	false = usuario desactivado. Default true

MÉTODOS — API de Service/Repository

Firma	Retorno	Descripción
createUsuario(Usuario)	Usuario	Crea cuenta de usuario
getUsuarioById(idUsuario)	Usuario	Busca por ID
updateUsuario(Usuario)	Usuario	Actualiza datos
getAllUsuario()	List<Usuario>	Lista todos los usuarios
deleteUsuarioById(id)	void	Elimina usuario
getModulosPermitidos(rol)	List<Modulo>	Módulos a los que tiene acceso según su rol

RELACIONES

- Usuario usa → RolUsuario (enum)
- Usuario 1 → 1 Tecnico (opcional, lado dueño de la FK)
- Usuario referenciado desde OrdenReparacion, HistorialEtapas, Observacion, Valoracion

Control de acceso

Modulo

Catálogo de pantallas/secciones del sistema. Agregar una restricción de acceso nueva es sólo una fila aquí, sin redespiegue.

Cambios respecto a la versión anterior

- Longitudes: codigo 255 → **10**, nombre 255 → **100**.
- Con 10 caracteres, códigos como **DETALLE_OT** caben justo. Un código más descriptivo (ej: VALORACION_DETALLE) ya no entra.

ATRIBUTOS

Nombre	Java	MySQL	Long.	Restricción	Descripción
idModulo	Long	BIGINT	20	PK	Identificador único
codigo	String	VARCHAR	10	NOT NULL, UNIQUE	Clave programática: BACKLOG, VALORACION, DETALLE_OT...
nombre	String	VARCHAR	100	NOT NULL	Nombre visible en la interfaz

MÉTODOS — API de Service/Repository

Firma	Retorno	Descripción
createModulo(Modulo)	Modulo	Crea módulo
getModuloById(idModulo)	Modulo	Busca por ID
updateModulo(Modulo)	Modulo	Actualiza datos
getAllModulo()	List<Modulo>	Lista todos los módulos
deleteModuloById(idModulo)	void	Elimina módulo

RELACIONES

- Modulo 1 → N RolModulo (mapeada: Modulo.rolesModulo)

RolModulo

Asignación de permisos: qué rol puede acceder a qué módulo. Spring Security consulta esta tabla (cacheada) vía JWT para autorizar cada request.

ATRIBUTOS

Nombre	Java	MySQL	Long.	Restricción	Descripción
idRolModulo	Long	BIGINT	20	PK	Identificador único
rol	RolUsuario	VARCHAR	20	NOT NULL	Rol al que se asigna el permiso
modulo (id_modulo)	Long	BIGINT	20	FK NOT NULL	Módulo al que se otorga acceso

MÉTODOS — API de Service/Repository

Firma	Retorno	Descripción
createRolModulo(RolModulo)	RolModulo	Asigna acceso
getModuloByRol(rol)	List<Modulo>	Módulos permitidos para el rol
deleteRolModulo(idRolModulo)	void	Revoca acceso

RELACIONES

- RolModulo N → 1 Modulo
- RolModulo usa → RolUsuario (enum)

Núcleo operativo

OrdenReparacion

Tabla central del sistema. Toda la trazabilidad parte de aquí. Contiene el puntero rápido a la etapa actual y todas las fechas clave del ciclo de vida.

Cambios respecto a la versión anterior

- **Los tres métodos de negocio salieron de la entidad.** cambiarEtapa(), asignarTecnico() y calcularDiasEnTaller() se reescriben en el Service; la entidad quedó sólo con campos y accesores.
- Las FK se mapean como objetos (Vehiculo, Cliente, Usuario, Tecnico, Etapas) con @JoinColumn, no como Long. La columna SQL va entre paréntesis.

ATRIBUTOS

Nombre	Java	MySQL	Long.	Restricción	Descripción
idOrdenReparacion	Long	BIGINT	20	PK	Consecutivo autoincremental
vehiculo (id_vehiculo)	Long	BIGINT	20	FK NOT NULL	Vehículo en reparación
cliente (id_cliente)	Long	BIGINT	20	FK NOT NULL	Propietario del vehículo
usuarioCreador (id_usuario_creador)	Long	BIGINT	20	FK NOT NULL	Usuario que abrió la orden
tecnicoResponsable (id_tecnico_responsable)	Long	BIGINT	20	FK [0..1]	Técnico responsable general de la OT
etapaActual (id_etapa_actual)	Long	BIGINT	20	FK NOT NULL	Puntero rápido a la etapa actual (backlog)
estado	Estado	VARCHAR	20	NOT NULL	ACTIVA CERRADA CANCELADA. Default ACTIVA
fechaIngresoCotizar	LocalDate	DATE	—	NOT NULL	Fecha en que entra a valoración
fechaIngresoReparacion	LocalDate	DATE	—	[0..1]	Fecha de inicio de reparación
fechaDeEntregaEstimada	LocalDate	DATE	—	[0..1]	Fecha estimada de entrega al cliente
fechaDeEntregaReal	LocalDate	DATE	—	[0..1]	Fecha real de entrega
diasEstimadoEntrega	Byte	TINYINT	3	[0..1]	Días estimados para completar. Máximo 127
createAt	LocalDateTime	DATETIME	—	NOT NULL	La asigna el Service
updateAt	LocalDateTime	DATETIME	—	NOT NULL	La asigna el Service

MÉTODOS — API de Service/Repository

Firma	Retorno	Descripción
createOrdenReparacion(OrdenReparacion)	OrdenReparacion	Abre nueva orden
getOrdenReparacionById(id)	OrdenReparacion	Busca por ID
getOrdenReparacionByPlaca(placa)	OrdenReparacion	Busca OT activa por placa — usado en la búsqueda del sistema
updateOrdenReparacion(OrdenReparacion)	OrdenReparacion	Actualiza datos generales
getAllOrdenReparacion()	List<OrdenReparacion>	Lista todas las órdenes
deleteOrdenReparacionById(id)	void	Elimina orden
cambiarEtapa(idOrden, idEtapa, idUsuario)	void	En el Service, @Transactional. Actualiza etapaActual, inserta en HistorialEtapas y cierra/abre fechas en OrdenEtapaFecha
asignarTecnico(idOrden, idTecnico)	void	En el Service. Asigna técnico responsable general
calcularDiasEnTaller(idOrden)	int	En el Service. Días transcurridos desde fechaIngresoCotizar

REGLAS DE NEGOCIO

- El cambio de etapa siempre va en una sola transacción: actualiza etapaActual + inserta HistorialEtapas + cierra fechaFin de la etapa anterior + abre fechaInicio de la etapa nueva.
- fechaFin de una etapa = fechaInicio de la etapa siguiente. NUNCA se marca manualmente desde la UI.
- diasEstimadoEntrega es Byte: si una reparación puede pasar de 127 días, hay que subir el tipo a Short.

RELACIONES

- OrdenReparacion N → 1 Vehiculo
- OrdenReparacion N → 1 Cliente
- OrdenReparacion N → 1 Usuario (usuarioCreador)
- OrdenReparacion N → 1 Tecnico (tecnicoResponsable, nullable)
- OrdenReparacion N → 1 Etapas (etapaActual)
- OrdenReparacion 1 → N HistorialEtapas (mapeada)
- OrdenReparacion 1 → N OrdenEtapaFecha (mapeada)
- OrdenReparacion 1 → 1 Valoracion (UNIQUE) (mapeada)
- OrdenReparacion 1 → N Observacion (sólo el lado dueño)
- OrdenReparacion 1 → N Notificaciones (sólo el lado dueño)

HistorialEtapas

Log de auditoría inmutable. Cada arrastre en el backlog inserta aquí. Es el gatillo que dispara las notificaciones automáticas al cliente. Sólo se insertan registros, nunca se actualizan.

Cambios respecto a la versión anterior

- **El campo comentario no existe en la entidad.** La versión anterior de esta guía lo listaba como VARCHAR(255) [0..1], pero nunca se implementó: la observación al mover la tarjeta se guarda en la tabla **Observacion**, que es la que usa el modal del backlog.

ATRIBUTOS

Nombre	Java	MySQL	Long.	Restricción	Descripción
idHistorialEtapas	Long	BIGINT	20	PK	Identificador único
ordenReparacion (id_orden_reparacion)	Long	BIGINT	20	FK NOT NULL	Orden a la que pertenece
etapa (id_etapa)	Long	BIGINT	20	FK NOT NULL	A qué etapa se movió
usuario (id_usuario)	Long	BIGINT	20	FK NOT NULL	Quién movió la tarjeta
fechaCambio	LocalDateTime	DATETIME	—	NOT NULL	Momento exacto del cambio — necesita hora, no sólo fecha

MÉTODOS — API de Service/Repository

Firma	Retorno	Descripción
createHistorialEtapas(HistorialEtapas)	HistorialEtapas	Inserta registro (sólo insert, nunca update)
getHistorialEtapasById(id)	HistorialEtapas	Busca por ID
getAllByOrden(idOrdenReparacion)	List<HistorialEtapas>	Historial completo de movimientos de una orden

RELACIONES

- HistorialEtapas N → 1 OrdenReparacion
- HistorialEtapas N → 1 Etapas
- HistorialEtapas N → 1 Usuario

OrdenEtapaFecha

Timeline por etapa. Una fila por etapa por orden (UNIQUE). Alimenta la vista de «Proceso de reparación». fechaFin es AUTOMÁTICA: la marca el sistema al pasar a la siguiente etapa, nunca manualmente.

ATRIBUTOS

Nombre	Java	MySQL	Long.	Restricción	Descripción
idOrdenEtapaFecha	Long	BIGINT	20	PK	Identificador único
ordenReparacion (id_orden_reparacion)	Long	BIGINT	20	FK NOT NULL	Orden a la que pertenece
etapa (id_etapa)	Long	BIGINT	20	FK NOT NULL	Etapa del timeline
tecnico (id_tecnico)	Long	BIGINT	20	FK [0..1]	Técnico que trabajó ESTA etapa puntual
fechaInicio	LocalDateTime	DATETIME	—	[0..1]	Se marca al entrar a la etapa (o al dejar observación sin fecha)
fechaFin	LocalDateTime	DATETIME	—	[0..1]	Se marca AUTOMÁTICAMENTE al pasar a la siguiente etapa
completada	Boolean	BOOLEAN	1	NOT NULL	true = etapa finalizada. Default false

MÉTODOS — API de Service/Repository

Firma	Retorno	Descripción
createOrdenEtapaFecha(OrdenEtapaFecha)	OrdenEtapaFecha	Crea fila de seguimiento
updateOrdenEtapaFecha(OrdenEtapaFecha)	OrdenEtapaFecha	Actualiza fechas o técnico
getOrdenEtapaFechaById(id)	OrdenEtapaFecha	Busca por ID
getAllByOrden(idOrdenReparacion)	List<OrdenEtapaFecha>	Timeline completo de la orden
marcarFechaInicio(idOrden, idEtapa)	void	Marca fechaInicio = NOW()
marcarFechaCompletada(idOrden, idEtapa)	void	Marca completada = true
asignarTecnicoEtapa(id, idTecnico)	void	Asigna técnico a esta etapa puntual

REGLAS DE NEGOCIO

- UNIQUE(id_orden_reparacion, id_etapa) — una sola fila por etapa por orden.
- fechaFin NUNCA se marca desde la UI. La marca el Service al cerrar la etapa anterior.
- El técnico se asigna aquí (no en OrdenReparacion): es la base del futuro sistema de pagos por trabajo realizado.

RELACIONES

- OrdenEtapaFecha N → 1 OrdenReparacion
- OrdenEtapaFecha N → 1 Etapas
- OrdenEtapaFecha N → 1 Tecnico (nullable)

Valoración

Valoracion

Hoja de valoración de daños. Una sola por orden (UNIQUE). Se edita, nunca se duplica. cargadaCesvi registra si ya fue subida a CESVI Colombia (plataforma externa).

Cambios respecto a la versión anterior

- descripcionGeneral 255 → **500**.
- **Se quitó CascadeType.ALL y orphanRemoval** de detalles e imagenes. Quitar un hijo de la lista en memoria ya no lo borra de la base.
- **marcarCargadaCesvi()** salió de la entidad y pasa al Service.

ATRIBUTOS

Nombre	Java	MySQL	Long.	Restricción	Descripción
idValoracion	Long	BIGINT	20	PK	Identificador único
ordenReparacion (id_orden_reparacion)	Long	BIGINT	20	FK NOT NULL, UNIQUE	Sólo una valoración por orden
usuario (id_usuario)	Long	BIGINT	20	FK NOT NULL	Quién elaboró la valoración
descripcionGeneral	String	VARCHAR	500	[0..1]	Descripción general del siniestro
costoEstimado	BigDecimal	DECIMAL	12,2	[0..1]	Costo estimado total
cargadaCesvi	boolean	BOOLEAN	1	NOT NULL	true = ya subida a CESVI Colombia. Default false
cargadaCesviAt	LocalDateTime	DATETIME	—	[0..1]	Momento en que se marcó como cargada en CESVI
createAt	LocalDateTime	DATETIME	—	NOT NULL	La asigna el Service
updateAt	LocalDateTime	DATETIME	—	NOT NULL	La asigna el Service

MÉTODOS — API de Service/Repository

Firma	Retorno	Descripción
createValoracion(Valoracion)	Valoracion	Crea valoración (sólo si no existe para esa orden)
updateValoracion(Valoracion)	Valoracion	Edita valoración existente
getValoracionByOrden(idOrden)	Valoracion	Busca la valoración de una orden — retorna null si no existe
cargarValoracion(idValoracion)	void	Marca la valoración como lista (disponible en la ficha de la orden)
marcarCargadaCesvi(idValoracion)	void	En el Service. El usuario confirma que ya subió las fotos/hoja a CESVI
generarHojaPdf(idValoracion)	byte[]	Genera PDF al vuelo — no se guarda en disco

REGLAS DE NEGOCIO

- UNIQUE(id_orden_reparacion) — un solo registro por orden. El endpoint usa UPSERT.
- El PDF y el ZIP de fotos se generan al vuelo con cada descarga, nunca se guardan como archivos fijos.
- cargadaCesvi se cambia manualmente desde el botón en la lista de valoraciones, no automáticamente.
- **Sin cascada:** al borrar una valoración, el Service debe borrar primero sus ValoracionDetalle y ValoracionImagen por su propio repositorio.

RELACIONES

- Valoracion 1 → 1 OrdenReparacion (UNIQUE, lado dueño de la FK)
- Valoracion 1 → N ValoracionDetalle (mapeada, sin cascada)
- Valoracion 1 → N ValoracionImagen (mapeada, sin cascada)
- Valoracion N → 1 Usuario

ValoracionDetalle

Cada pieza evaluada en la valoración. gravedad es nullable cuando accion = SUSTITUCION.

Cambios respecto a la versión anterior

- Se eliminó el CHECK ck_detalle_gravedad del @Table. El DDL ya no impide guardar una SUSTITUCION con gravedad, ni una REPARACION sin ella.
- El cascade delete desde Valoracion también se quitó.

ATRIBUTOS

Nombre	Java	MySQL	Long.	Restricción	Descripción
idValoracionDetalle	Long	BIGINT	20	PK	Identificador único
valoracion (id_valoracion)	Long	BIGINT	20	FK NOT NULL	Valoración a la que pertenece
pieza	String	VARCHAR	255	NOT NULL	Ej: Puerta delantera derecha
accion	Accion	VARCHAR	20	NOT NULL	REPARACION SUSTITUCION
gravedad	Gravedad	VARCHAR	20	[0..1]	LEVE MEDIO FUERTE — sólo si accion = REPARACION
observacion	String	VARCHAR	255	[0..1]	Nota específica por pieza
costo	BigDecimal	DECIMAL	12,2	[0..1]	Costo individual de esa pieza

MÉTODOS — API de Service/Repository

Firma	Retorno	Descripción
createValoracionDetalle(ValoracionDetalle)	ValoracionDetalle	Agrega pieza a la valoración
getAllByIdValoracion(idValoracion)	List<ValoracionDetalle>	Lista piezas de una valoración
updateValoracionDetalle(ValoracionDetalle)	ValoracionDetalle	Actualiza la pieza
deleteValoracionDetalleById(id)	void	Elimina la pieza

REGLAS DE NEGOCIO

- Regla: (accion = REPARACION AND gravedad IS NOT NULL) OR (accion = SUSTITUCION AND gravedad IS NULL).
- Esa regla ya no la impone la base de datos: la tiene que validar el Service antes de guardar.
- En el frontend: el selector de gravedad se oculta cuando la acción es SUSTITUCION.

RELACIONES

- ValoracionDetalle N → 1 Valoracion (sin cascade delete)

ValoracionImagen

Fotos de soporte de la valoración. Las URLs deben ser públicas (https) para que la API de WhatsApp y CESVI puedan descargarlas.

Cambios respecto a la versión anterior

- url 255 → **2083** (el límite práctico de URL de los navegadores) y descripcion 255 → **300**.
- Con utf8mb4, MySQL **no puede indexar** una columna de 2083 completa: no se le puede poner UNIQUE sin usar prefijo de índice.
- El cascade delete desde Valoracion se quitó.

ATRIBUTOS

Nombre	Java	MySQL	Long.	Restricción	Descripción
idValoracionImagen	Long	BIGINT	20	PK	Identificador único
valoracion (id_valoracion)	Long	BIGINT	20	FK NOT NULL	Valoración a la que pertenece
url	String	VARCHAR	2083	NOT NULL	URL pública https del archivo
descripcion	String	VARCHAR	300	[0..1]	Descripción opcional de la foto
createAt	LocalDateTime	DATETIME	—	NOT NULL	La asigna el Service

MÉTODOS — API de Service/Repository

Firma	Retorno	Descripción
createValoracionImagen(ValoracionImagen)	ValoracionImagen	Registra la foto
getAllByIdValoracion(idValoracion)	List<ValoracionImagen>	Lista las fotos de una valoración
deleteByIdValoracionImagen(id)	void	Elimina la foto
descargarTodasZip(idValoracion)	byte[]	Genera ZIP al vuelo con todas las fotos — para subir a CESVI

RELACIONES

- ValoracionImagen N → 1 Valoracion (sin cascade delete)

Observaciones

Observacion

Observaciones estructuradas (inspirado en Orbika). Se ingresan desde dos puntos: el modal inline del backlog (al mover la tarjeta) o la ficha de detalle. Ambas rutas escriben en esta misma tabla.

Cambios respecto a la versión anterior

- nota 255 → **500**.
- Es la **única relación del modelo que conserva** cascade = ALL + orphanRemoval (hacia sus imágenes).

ATRIBUTOS

Nombre	Java	MySQL	Long.	Restricción	Descripción
idObservacion	Long	BIGINT	20	PK	Identificador único
ordenReparacion (id_orden_reparacion)	Long	BIGINT	20	FK NOT NULL	Orden a la que pertenece
usuario (id_usuario)	Long	BIGINT	20	FK NOT NULL	Quién dejó la observación
etapa (id_etapa)	Long	BIGINT	20	FK [0..1]	Etapa al momento de la nota
ubicacion	Ubicacion	VARCHAR	20	NOT NULL	EN_TALLER FUERA_DE_TALLER
estadoRepuestos	Repuestos	VARCHAR	20	[0..1]	COMPLETOS PENDIENTES
nota	String	VARCHAR	500	NOT NULL	Texto de la observación
visibleCliente	boolean	BOOLEAN	1	NOT NULL	true = se notifica al cliente por WhatsApp. Default true
createAt	LocalDateTime	DATETIME	—	NOT NULL	La asigna el Service

MÉTODOS — API de Service/Repository

Firma	Retorno	Descripción
createObservacion(Observacion)	Observacion	Crea observación. Si la etapa no tiene fechaInicio, la marca automáticamente (misma transacción)
updateObservacion(Observacion)	Observacion	Actualiza la observación
getObservacionById(id)	Observacion	Busca por ID
getAllObservacion(idOrdenReparacion)	List<Observacion>	Historial de observaciones de la orden
deleteObservacionById(id)	void	Elimina la observación

REGLAS DE NEGOCIO

- Si visibleCliente = true, el servicio de notificaciones envía el mensaje + las fotos adjuntas por WhatsApp.
- Si la etapa referenciada tiene fechaInicio = NULL al crear la observación, el Service la marca con NOW() en la misma transacción.
- ubicacion y estadoRepuestos viven aquí, no en OrdenReparacion: son el estado *al momento de dejar la nota*.

RELACIONES

- Observacion N → 1 OrdenReparacion
- Observacion N → 1 Usuario
- Observacion N → 1 Etapas (nullable)
- Observacion 1 → N ImagenObservacion (mapeada, CON cascada)
- Observacion 1 → N Notificaciones (como origen)

ImagenObservacion

Fotos adjuntas a una observación. Se envían al cliente junto con el texto por WhatsApp. La URL debe ser pública (https).

Nota de consistencia

- `url` aquí quedó en 255, mientras que `ValoracionImagen.url` subió a 2083. Las dos guardan el mismo tipo de dato (URL pública https), así que conviene igualarlas.

ATRIBUTOS

Nombre	Java	MySQL	Long.	Restricción	Descripción
<code>idImagenesObservacion</code>	Long	BIGINT	20	PK	Identificador único. El nombre va en plural, según la guía original
<code>observacion (id_observacion)</code>	Long	BIGINT	20	FK NOT NULL	Observación a la que pertenece
<code>url</code>	String	VARCHAR	255	NOT NULL	URL pública https
<code>createAt</code>	LocalDateTime	DATETIME	—	NOT NULL	La asigna el Service

MÉTODOS — API de Service/Repository

Firma	Retorno	Descripción
<code>createImagenObservacion(ImagenObservacion)</code>	<code>ImagenObservacion</code>	Registra la foto
<code>getAllByObservacion(idObservacion)</code>	<code>List<ImagenObservacion></code>	Lista las fotos de una observación
<code>deleteImagenById(id)</code>	<code>void</code>	Elimina la foto

RELACIONES

- `ImagenObservacion` N → 1 `Observacion` (CASCADE DELETE)

Notificaciones

Notificaciones

Log de todos los mensajes enviados por WhatsApp. Permite debug, reintentos y auditoría de qué llegó al cliente y cuándo.

Cambios respecto a la versión anterior

- **marcarEnviada()** y **marcarFallida()** salieron de la entidad y pasan al Service.
- (*) Cuatro columnas quedaron **sin length explícito** (255 por default). **contenidoEnviado** es el caso urgente: un mensaje de WhatsApp se pasa fácil de 255 caracteres y MySQL lo truncaría. Evaluar TEXT.

ATRIBUTOS

Nombre	Java	MySQL	Long.	Restricción	Descripción
idNotificacion	Long	BIGINT	20	PK	Identificador único
ordenReparacion (id_orden_reparacion)	Long	BIGINT	20	FK NOT NULL	Orden a la que pertenece
cliente (id_cliente)	Long	BIGINT	20	FK NOT NULL	Receptor del mensaje
observacion (id_observacion)	Long	BIGINT	20	FK [0..1]	Origen del mensaje si aplica
eventoDisparador	String	VARCHAR	255 *	NOT NULL	CAMBIO_ETAPA NUEVA_OBSERVACION. Sin length explícito
canal	String	VARCHAR	255 *	NOT NULL	WHATSAPP. Default en la entidad. Sin length explícito
contenidoEnviado	String	VARCHAR	255 *	NOT NULL	Texto exacto enviado. Sin length explícito
imagenesEnviadas	String (JSON)	JSON	—	[0..1]	Array JSON de URLs de fotos enviadas
estadoEnvio	EstadoEnvio	VARCHAR	20	NOT NULL	PENDIENTE ENVIADO FALLIDO. Default PENDIENTE
fechaEnvio	LocalDateTime	DATETIME	—	[0..1]	Momento del envío efectivo
errorDetalle	String	VARCHAR	255 *	[0..1]	Detalle del error si estadoEnvio = FALLIDO. Sin length explícito
createAt	LocalDateTime	DATETIME	—	NOT NULL	La asigna el Service

MÉTODOS — API de Service/Repository

Firma	Retorno	Descripción
createNotificacion(Notificaciones)	Notificaciones	Registra el mensaje
getNotificacionesById(id)	Notificaciones	Busca por ID
getAllByOrden(idOrdenReparacion)	List<Notificaciones>	Mensajes de una orden
reintentarEnvio(idNotificacion)	void	Reintenta el envío si estadoEnvio = FALLIDO
marcarEnviada(idNotificacion)	void	En el Service. estadoEnvio = ENVIADO + fechaEnvio = NOW()
marcarFallida(idNotificacion, error)	void	En el Service. estadoEnvio = FALLIDO + errorDetalle

REGLAS DE NEGOCIO

- El listener de notificaciones es **@Async**: un fallo en WhatsApp no rompe la operación principal ni hace rollback de la transacción.
- La API de WhatsApp (Twilio/Meta) necesita una URL pública https para las imágenes — no acepta archivos directos.
- Un mensaje con 3 fotos genera: 1 mensaje de texto + 3 mensajes de media (una foto cada uno).

RELACIONES

- Notificaciones N → 1 OrdenReparacion
- Notificaciones N → 1 Cliente
- Notificaciones N → 1 Observacion (nullable)

PlantillaMensaje

Plantillas de texto para los mensajes automáticos. Soportan variables interpoladas como {{cliente}}, {{placa}}, {{etapa}}.

Pendiente

- **contenidoTemplate** sigue en 255, igual que en la guía original. Una plantilla con variables interpoladas se pasa de ahí con facilidad: evaluar subirla a TEXT.

ATRIBUTOS

Nombre	Java	MySQL	Long.	Restricción	Descripción
idPlantillaMensaje	Long	BIGINT	20	PK	Identificador único
evento	String	VARCHAR	255	NOT NULL	CAMBIO_ETAPA NUEVA_OBSERVACION
canal	String	VARCHAR	255	NOT NULL	Canal de envío. Default WHATSAPP en la entidad
contenidoTemplate	String	VARCHAR	255	NOT NULL	Ej: Hola {{cliente}}, tu vehículo {{placa}} pasó a {{etapa}}.
activo	boolean	BOOLEAN	1	NOT NULL	false = plantilla desactivada. Default true

MÉTODOS — API de Service/Repository

Firma	Retorno	Descripción
createPlantilla(PlantillaMensaje)	PlantillaMensaje	Crea la plantilla
getPlantillaByEvento(evento)	PlantillaMensaje	Usada por el listener para construir el mensaje
updatePlantilla(PlantillaMensaje)	PlantillaMensaje	Edita la plantilla
getAllPlantilla()	List<PlantillaMensaje>	Lista todas las plantillas

RELACIONES

- PlantillaMensaje consultada por el servicio de notificaciones (sin FK directa)